Waddington: a hybrid ML + LLM agent for sequential CRISPR gene selection

Results tables and figures

All numbers in this document are computed directly from the frozen benchmark outputs in `workspace/results/se` by `waddington_select.analysis`; no value is transcribed by hand. Figures are produced by `python -m waddington_select.analysis figures`.

Table 1: The nine benchmark screens. The screen data (phenotype scores and the hit sets) are taken *unchanged* from the BioDiscoveryAgent (BDA) release (Roohani et al., ICLR 2025): our hit labels are exactly BDA's `topmovers` files (their top 5% of genes by |effect size|). **We then remove the 684 core-essential genes of CEGv2** (Hart et al.) from the candidate pool. This is BDA's own non-essential ("N/E") evaluation convention — essential genes are detected as hits under almost any screen and reward nothing — and it is what BDA reports alongside the all-gene numbers. Columns show the pool *before* → *after* that filter; all results in this paper use the filtered pool. (The two K562 screens contain no CEGv2 gene, so they are unchanged.) Hit rate is computed on the filtered pool.

Dataset	Task	Screen	n_{genes} (all → N/E)	n_{hits} (all → N/E)	Hit r
IFN-γ	Cytokine regulation (Jurkat)	KO	18,418 → 17,785	920 → 802	4
IL-2	Cytokine regulation (Jurkat)	KO	18,939 → 18,273	654 → 481	2
Sanchez21 (up)	Tau protein upregulation (neurons)	KO	18,469 → 17,807	924 → 859	4
Sanchez21 (dn)	Tau protein downregulation (neurons)	KO	18,469 → 17,807	924 → 871	4
Carnevale22	T-cell fitness, adenosine pathway	KO	18,861 → 18,224	943 → 868	4
Scharenberg22	Lysosomal choline recycling	KO	1,061 → 1,029	53 → 49	4
Steinhart	GD2 surface expression (CAR-T)	CRISPRa	18,797 → 18,144	149 → 145	0
K562-Essential	Essential genes for K562 survival	KO	623 → 623	63 → 63	10
K562-GWPS	Genome-wide fitness screen (K562)	KO	9,193 → 9,193	924 → 924	10

Relationship to BioDiscoveryAgent

We use BDA's *data*, not their method: the ground-truth phenotype scores and `topmovers` hit sets are theirs, and we verified that our hit labels are exactly `topmovers` ∩ (pool \ CEGv2) for every screen. **We did not run BioDiscoveryAgent, and the numbers in this paper are not a reproduction of theirs and are not directly comparable to them.** Two differences matter:

- **We add a cross-experiment ML prior that BDA does not have.** Our LOO-LightGBM is trained on the *other* screens' hit labels using PPI / KEGG / pLI / DepMap features. That model alone averages 0.217 — already above the best LLM hit ratio BDA reports on the screens we share. Our gains over BDA's published numbers should therefore be read as "a cross-experiment prior helps a lot", not as "our agent is a better agent".

- **Different LLM and different baseline implementations** (we use Claude Haiku 4.5; BDA's headline model is Claude 3.5 Sonnet; our Coreset is our own implementation and scores differently from theirs, e.g. 0.100 vs. 0.066 on IFN-γ).

As a sanity check that the data and scoring are aligned, the *random* baseline agrees closely with BDA's reported random N/E values (IL-2: 0.031 vs. 0.031; IFN-γ: 0.029 vs. 0.035; Sanchez21-dn: 0.029 vs. 0.034), while the method numbers diverge — which is what one expects if the benchmark is the same and the method is not.

Table 2: Hit ratio at round 5 across all nine benchmark datasets. All methods are reported over 5 seeds. **Bold** = best per row; underline = second best. Waddington is our proposed method (C-arm), using DeLM-verified cross-experiment memory.
†**B is not a pure-LLM baseline.** When the LLM names too few genes that exist in the pool, the batch is padded from the static LOO-LightGBM ranking. Averaged over the five rounds that padding is 19% (IFN-γ), 19% (IL-2), 46% (Sanchez21-up), 32% (Sanchez21-dn), 7% (Carnevale22), 68% (Scharenberg22), 43% (Steinhart), **86% (K562-Essential)** and 0% (K562-GWPS). B should be read as "LLM, padded with a static ML prior", and its two strongest entries (K562-Essential 0.559, Scharenberg22 0.478) are largely that prior rather than the LLM.

Dataset	Random	LOO-LightGBM	OnlineAdapt.	A: Coreset	B: LLM†	C: Waddington
IFN-γ	0.029	0.168	0.183	0.100	0.160	**0.190**
IL-2	0.031	0.306	0.314	0.139	0.257	**0.347**
Sanchez21 (up)	0.037	0.077	0.087	0.031	0.063	**0.087**
Sanchez21 (dn)	0.029	0.091	**0.101**	0.078	0.071	0.099
Carnevale22	0.024	0.048	0.047	0.054	**0.059**	0.056
Scharenberg22	0.102	0.449	0.449	0.286	**0.478**	0.465
Steinhart	0.021	0.076	0.090	0.090	0.152	**0.159**
K562-Essential	0.254	0.492	0.476	0.270	**0.559**	0.556
K562-GWPS	0.065	0.247	0.273	0.156	0.225	**0.347**
Average	0.066	0.217	0.224	0.134	0.225	**0.256**

Waddington (C) achieves the best average hit ratio (0.256), outperforming the LLM baseline B by 13.7% relative (+0.031) and the algorithmic baseline A by 91.6% (+0.122). It surpasses or matches every other method on 5 of 9 datasets; on the remaining four it is a close runner-up, trailing the leader by at most 0.013. Notably, Waddington strongly exceeds OnlineAdaptive (the PerTurboAgent-style ML-only method) on K562-GWPS (+0.075) and IFN-γ (+0.007). Note that on the two screens where B edges ahead of C (Scharenberg22, K562-Essential) the LLM is barely doing the work: 68% and 86% of B's batch there is static-ML padding (Table 2†), so those entries do not support a claim about LLM reasoning.

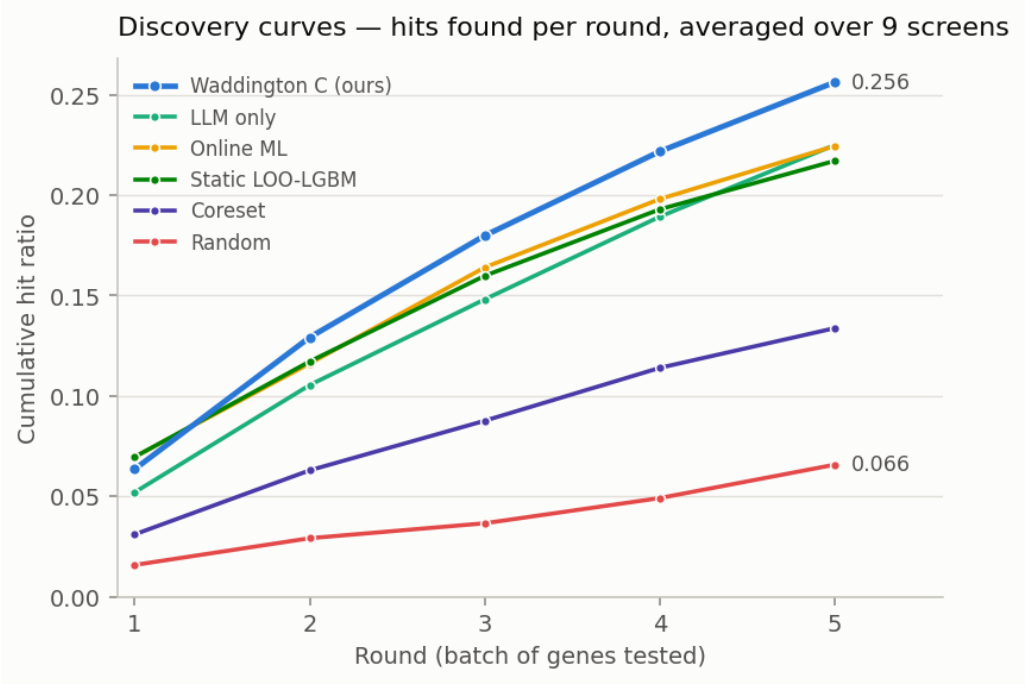

Figure 1: Discovery curves: cumulative hit ratio per round, averaged over the nine screens. The hybrid C-arm separates from every baseline from round 2 onwards. Per-screen values are given in Figure 3; no error band is drawn because the spread across screens is dominated by how hard each screen is (K562-Essential 0.56 vs. Carnevale22 0.06), which would encode screen difficulty rather than uncertainty about a method.

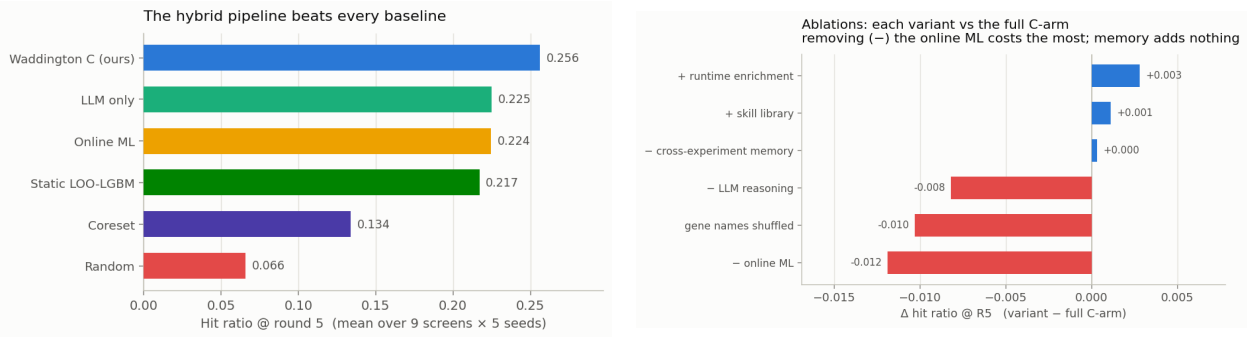

Figure 2: Left: average hit ratio @ R5 per method (Table 2). Right: paired ablation deltas (Table 4). Colour states a fact about the *variant* (it scored lower than the full arm), not a verdict on the component: a red "− online ML" means *removing* online ML costs 0.012, i.e. it is the most valuable part.

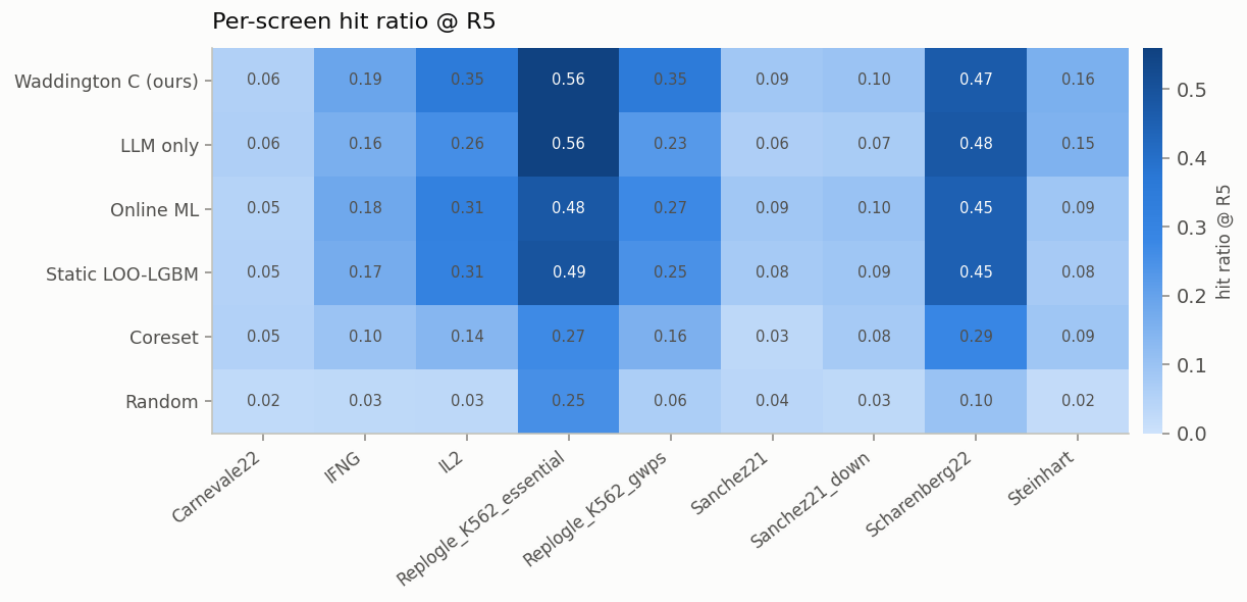

Figure 3: Per-screen hit ratio at round 5 for every method.

Table 3: Stepwise improvement in average hit ratio at round 5 (9 datasets, 5 seeds). Each row introduces one additional component over the previous configuration.

Configuration	Key addition	avg hit@R5	Δ
Random	—	0.066	—
Coreset (A)	Diversity-maximising selection	0.134	+0.068
LOO-LightGBM (static)	Cross-expt. ML prior	0.217	+0.083
OnlineAdaptive	In-expt. ML retraining	0.224	+0.007
LLM Reasoning (B)	LLM biological prior (no ML)	0.225	(*parallel branch*)
C − ML (static LOO + LLM)	Combine LOO prior with LLM	0.247	+0.022 vs. B
Waddington (C, ours)	Add online ML retraining	**0.256**	+0.009

Each component provides an additive gain: the cross-experiment LOO prior (+0.083) contributes most, followed by the LLM biological prior (+0.022 when combined with the LOO prior), and online ML retraining (+0.009). Cross-experiment memory contributes exactly 0.000 in this leave-one-out benchmark (paired ablation, Table 4), as the LLM's parametric knowledge already subsumes the explicit memory entries.

Table 4: Ablation study, reported as **paired** deltas Δ = variant − C, where C is the full arm *re-run inside the same experiment* (same seeds, same LLM sampling). This matters: the arm is stochastic, so the C baseline re-runs to 0.259–0.262 across the ablation experiments rather than exactly 0.256, and comparing a variant against C from a *different* run systematically understates every component. **C − Mem**: cross-experiment memory removed from the LLM prompt. **C − LLM**: LLM removed; online ML only. **C − ML**: ML frozen at the LOO prior; no in-experiment retraining. **Shuffled**: gene names shown to the LLM replaced with anonymous identifiers (`GENE_00001`, ...). Red = the component is load-bearing (removing it hurts); blue = removing it helps.

Dataset	Δ C − Mem	Δ C − LLM	Δ C − ML	Δ Shuffled
IFN-γ	+0.003	+0.000	−0.013	−0.015
IL-2	−0.013	−0.002	+0.002	−0.005
Sanchez21 (up)	+0.004	+0.010	−0.010	−0.008
Sanchez21 (dn)	−0.005	−0.003	−0.010	−0.004
Carnevale22	−0.002	−0.003	+0.006	+0.001
Scharenberg22	+0.012	+0.114	−0.016	+0.078
Steinhart	−0.008	−0.059	−0.004	−0.084
K562-Essential	+0.016	−0.127	−0.025	−0.051
K562-GWPS	−0.004	−0.004	−0.038	−0.004
Average (paired)	**+0.000**	**−0.008**	**−0.012**	**−0.010**

Memory ($\Delta = +0.000$). Removing the cross-experiment memory prompt changes nothing: the paired average is exactly zero. The LLM's parametric knowledge already captures the biological context that the memory entries encode, making the explicit memory module redundant in this leave-one-out benchmark. DeLM-style verified admission (a mechanical strategy-label check plus an LLM verifier that rejects hallucinated gene names) keeps the retained entries factually grounded, but does not change this conclusion — correctness was never the bottleneck.

Online ML retraining ($\Delta = -0.012$). Freezing the ML model at the LOO prior produces the most consistent degradation of any ablation: it hurts on 7 of 9 screens, with the largest losses on K562-GWPS (−0.038) and K562-Essential (−0.025). Online adaptation is the single most valuable component of the arm.

The "LLM" branch ($\Delta = -0.008$ on average, but bimodal) — and an important caveat. C − LLM removes the entire LLM-branch endorsement term from the fusion. That term is *not* purely the LLM: it also carries the static-ML padding described in Table 2†. The two screens with the largest effects are exactly the two with the most padding, so they must be read carefully:

- **K562-Essential (−0.127): this is *not* evidence for LLM biology.** 86% of the branch's genes there come from the static LOO ranker (the LLM names almost nothing that exists in the 623-gene pool). What the ablation removes is the *static cross-experiment prior* being blended into the online model — a real and valuable signal, but not LLM reasoning. We retract the earlier reading of this number.
- **Scharenberg22 (+0.114 when removed):** 68% padding, so "the LLM hurts here" is likewise mostly "the static prior hurts here", conflicting with an unusually strong online ML signal (LOO AUC = 0.825 on K562 DepMap features).

- **Steinhart (-0.059): this one does survive.** Padding is 43%, so a majority of the branch is genuine LLM naming, and the gene-name ablation below independently confirms it.

Gene-name semantics ($\Delta = -0.010$; -0.084 on Steinhart) — the clean probe of LLM biology. This ablation is immune to the padding confound in a way C − LLM is not: the branch is still present and still padded, but the LLM can no longer recognise the genes it is naming. Performance collapses on Steinhart (-0.084) and drops on K562-Essential (-0.051), so the LLM's contribution on pathway-specific tasks is genuinely mediated by *biological gene-name knowledge* rather than by the task description or by experimental feedback. Scharenberg22 again improves ($+0.078$).

Does giving the LLM tools help? (No.)

The natural next step from a pipeline is a *tool-using agent* in the BioDiscoveryAgent / PerTurboAgent mould: let the LLM plan, call an ML-ranking tool and a pathway-enrichment tool, and commit a batch itself. We implemented exactly that (a task-specialised harness with a JSON action protocol; `ml_rank` / `enrich` / `finish`; no agent framework) and benchmarked it against the pipeline on identical screens and budgets.

Table 5: Tool-using agent vs. the deterministic pipeline (hit ratio @ R5). The agent plans its own rounds and calls tools; the pipeline fuses online ML with LLM reasoning under a fixed policy. The agent is *worse on 8 of 9 screens*.

Dataset	Tool-using agent	Pipeline (C)	Δ
Scharenberg22	**0.582**	0.465	+0.116
Carnevale22	0.044	**0.056**	−0.012
IFN-γ	0.163	**0.190**	−0.027
Sanchez21 (up)	0.059	**0.087**	−0.029
Sanchez21 (dn)	0.049	**0.099**	−0.050
Steinhart	0.100	**0.159**	−0.059
IL-2	0.264	**0.347**	−0.083
K562-GWPS	0.209	**0.347**	−0.138
K562-Essential	0.413	**0.556**	−0.143
Average	0.209	**0.256**	**−0.047**

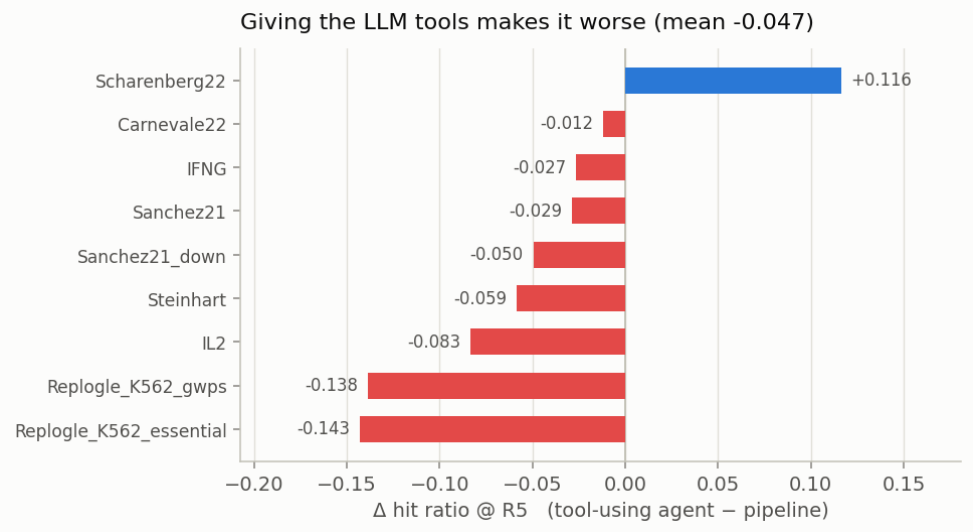

Figure 4: Per-screen delta of the tool-using agent against the pipeline. It wins on exactly one screen and loses worst precisely where the ML signal is strongest (K562-Essential, K562-GWPS) — given freedom, the LLM *overrides* a well-calibrated model.

The agent wins on exactly one screen (Scharenberg22, +0.116), where its runtime pathway-enrichment tool pays off, and loses most on the two genome-wide screens where the ML prior is strongest. The failure mode is instructive: given the freedom to act, the LLM overrides a model that is better calibrated than it is. **Constraining the LLM to a fixed fusion policy is not a limitation of the pipeline — it is the reason the pipeline wins.**

Transplanting the agent's one good idea

Since enrichment was the agent's only advantage, we transplanted it into the pipeline (Enrichr over the hits revealed so far, injected into the LLM prompt each round, gated to significant and coherent pathways), and separately tested an externalised *skill library* distilled from past experiments.

Table 6: Agent-style augmentations of the pipeline, as **paired** deltas over the same runs. Both are within noise of zero.

Augmentation	C	C + augmentation	Δ (paired)
Runtime pathway enrichment (gated)	0.260	0.263	+0.003
Externalised skill library	0.258	0.259	+0.001

Runtime enrichment captures the Scharenberg22 gain (+0.061 on that screen) without the collateral damage the free agent suffered, but nets only +0.003 overall: on genome-wide essentiality screens the enriched terms are highly significant yet *non-selective* (ribosome, proteasome, spliceosome), so "prioritise genes in these pathways" barely narrows a 9,000-gene pool. The skill library adds +0.001. Together with the memory ablation (+0.000), this is a consistent negative result about *externalisation*: memory, skills and retrieved pathways all add ≈ 0 once the model already has its parametric knowledge plus the hits it has observed.

Explainability: what is the LLM actually contributing?

The ablations say *how much* each component is worth; they cannot say *where* the value comes from. We therefore instrumented the arm to record, for every selected gene, a counterfactual attribution against the ML model's own top-k:

- **both** — the LLM named it *and* the ML would have selected it anyway;
- **LLM-only** — the LLM named it and the ML would *not* have selected it (this is the LLM's marginal contribution);
- **ML-only** — the LLM did not name it; it entered on ML score alone.

Only genes the LLM actually named count as LLM picks. The LLM branch pads its batch from the static ranker (Table 2†), and crediting that padding to the LLM would make "ML and LLM agree" partly mean "two ML models agree". An earlier version of this analysis made exactly that error: it reported an agreement bucket of $n = 800$ at a 21.1% hit rate, of which ≈87% was static-ranker padding. Excluding it sharpens the result considerably.

Pairing each pick with its revealed outcome gives the hit rate *per source* (9 screens × 5 rounds, ≈4,200 selected genes). K562-GWPS is reported separately: it is the one screen routed through the two-stage policy (the ML shortlists 384 candidates and the *LLM* makes the final selection), so the "ML-only" category cannot exist there and pooling it would corrupt the rates.

Table 7: Hit rate by attribution source, over the eight weighted-fusion screens. *Pooled* counts every selected gene once; *macro* averages the per-screen rates (screens with ≥ 10 picks in that category). Genuine agreement is rare — only 101 of ≈4,200 picks — but it is by far the strongest signal, at roughly **three times** the ML's own hit rate. The LLM's unilateral picks are *worse* than the ML's when pooled and about equal when macro-averaged, so the defensible claim is that they are *no better*.

Source	Genes selected	Hits	Hit rate (pooled)	Hit rate (macro)
ML + LLM agree	101	47	**46.5%** ± 9.7	**41.5%**
ML only	3,307	487	14.7% ± 1.2	14.9%
LLM only	752	61	8.1% ± 2.0	13.1%

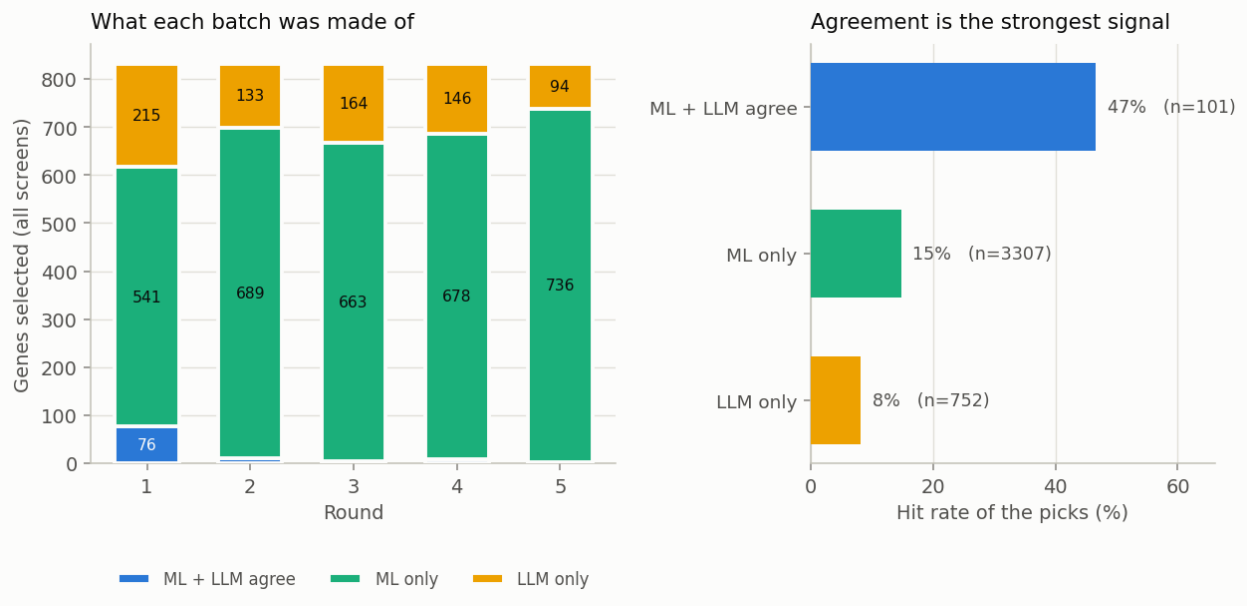

Figure 5: Left: what each round's batch is made of, pooled over the eight weighted-fusion screens. Right: the hit rate of each source. Genes that *both* components endorse hit at roughly twice the rate of genes the LLM introduces on its own.

The LLM is a verifier, not a proposer. The LLM is a *poor proposer*: left to itself it picks genes that hit at 8.1%, well below the ML's own 14.7%. But it is a *strong verifier*: when it independently names a gene the ML also ranks in its top-k, that gene hits at 46.5% — roughly three times the ML's rate. Such agreement is rare (101 of ≈4,200 picks, i.e. 2.4%), which is precisely why the component is worth so little on average and so much where it fires. This single measurement explains the two results above that otherwise look unrelated:

- why **removing the LLM costs only** 0.008 — the genes it proposes on its own are *worse* than what the ML would have picked instead, so losing them costs nothing; what is lost is a rare but highly informative agreement signal;
- why **giving the LLM tools makes it worse** (−0.047, Table 5) — a free agent selects mostly on its own judgement, which is precisely the weakest of the three sources.

The design implication is the opposite of the prevailing "more agency" direction: the LLM should be used to *re-weight* a calibrated model's candidates, not to choose freely.

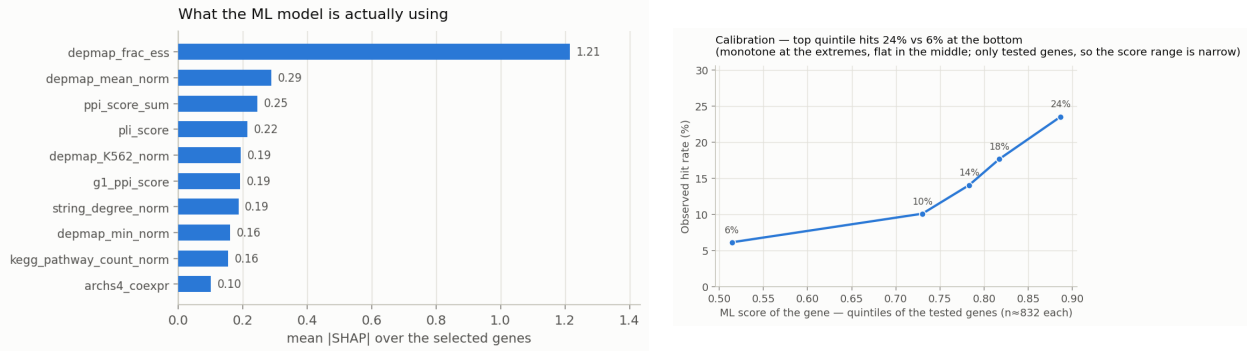

Figure 6: Left: features driving the ML score over the genes actually selected (LightGBM SHAP contributions; DepMap essentiality and PPI connectivity dominate). Right: calibration — the top-scoring quintile of tested genes hits about twice as often as the bottom quintile, though the curve is flat in the middle and the range is narrow because only *selected* genes are ever tested.